

Low-Depth Bootstrapping for Matrix-Native FHE

Rostin Shokri and Nektarios Georgios Tsoutsos

University of Delaware
{rostinsh, tsoutsos}@udel.edu

Abstract. Fully homomorphic encryption (FHE) enables computation directly on encrypted data and is increasingly researched for privacy-preserving machine-learning inference. Such workloads are dominated by matrix multiplication, which is not represented natively by conventional vector-oriented FHE schemes. CKKS is widely used for approximate encrypted inference because it supports packed real and complex arithmetic; however, matrix multiplication generally requires specific packing formats that are tailored to certain models, and involves many expensive ciphertext rotations. Depending on the matrix dimensions and packing strategy, these techniques can introduce substantial latency and memory overhead. The recently proposed Gentry–Lee (GL) scheme instead provides native slot-encoded matrix arithmetic through a multivariate ring construction while retaining approximate packed computation. Although promising, the prime- p GL instances considered here require prime-length NTTs and a three-dimensional DFT message encoding, whose efficient implementation has received limited study. Moreover, the original scheme does not specify a concrete bootstrapping procedure, which is required to support the deep computations used by large neural-network models.

In this work, we present a low-depth complex bootstrapping framework tailored to the GL scheme. Our construction combines a grouped, depth-1 GL StC, in which the X -axis inversion uses native plaintext–ciphertext matrix multiplication and the W -axis inversion uses a diagonal BSGS transform, with a slice-wise adaptation of SHIP to the Gaussian (X, W) algebra. The public masked-column candidates evaluate the complete W -axis monomial action and the fine X -axis residue, leaving only the coarse X -axis displacement for hidden HMux selection and eliminating encrypted W -axis selection. We further algebraically reduce the online plaintext-encoding requirement of the masked-column interface from the fully expanded candidate-family upper bound of $8\theta\varphi(p) + 2$ to $2\varphi(p) + 2$ plaintexts per Y -slice and optimize the Rader NTT through fused kernels and other arithmetic optimizations. Across matched CKKS configurations, GL achieves a $2.55\times$ – $3.12\times$ speedup in amortized per-slice StC latency and a $1.09\times$ – $1.16\times$ speedup in amortized per-slice complex-bootstrap latency.

Keywords: Fully Homomorphic Encryption · Gentry-Lee Scheme · Matrix Arithmetic · Bootstrapping · Privacy-Preserving Machine Learning

1 Introduction

Privacy-preserving machine learning [21, 28, 31] has received sustained attention, motivated in part by the growing availability of machine learning as a service (MLaaS), through which remotely hosted models are exposed via prediction APIs [19, 30], and by the need to protect sensitive inference inputs. Fully homomorphic encryption (FHE) [15] is one of the most compelling cryptographic approaches for this setting, as it enables computation directly on encrypted data without requiring decryption by the service provider. In a typical private inference workflow, a client encrypts its input under its secret key, the server evaluates the model over ciphertexts, and the client decrypts the prediction locally. This model offers strong confidentiality guarantees, but its practicality depends critically on the efficiency of the underlying FHE scheme and, in particular, on the cost of the ciphertext-refresh operation known as bootstrapping [16].

In leveled schemes based on ring learning with errors (RLWE) [24], such as CKKS [11] and GL [17], homomorphic evaluation progressively consumes the available noise, and modulus budget. Once a computation exhausts the modulus budget and noise, further computations become unavailable. Bootstrapping addresses this problem by restoring the modulus and applying a homomorphic modular reduction, and producing a new ciphertext with renewed computational depth. As a result, bootstrapping is a central building

block for deep homomorphic workloads, including modern neural networks with many layers, nonlinearities, and repeated linear algebra operations. As hardware accelerators and FHE algorithms continue to improve, enabling deeper and more expressive encrypted models, the cost of bootstrapping increasingly determines end-to-end system performance.

Many recent FHE-based private-inference systems use CKKS because it supports packed approximate arithmetic over real and complex values [9, 11, 22, 32]. Substantial work has studied the latency, precision, depth, and parallelism of CKKS bootstrapping [4, 6]. Matrix multiplication is a major computational kernel in many neural-network workloads. Although CKKS is primarily organized around packed vector arithmetic, matrix multiplication need not rely exclusively on rotation-based layouts and sequences of element-wise operations [1, 23]. Reduction-based constructions reduce plaintext–ciphertext and ciphertext–ciphertext matrix multiplication to a small number of plaintext modular matrix multiplications [2, 26], while the SinC construction improves batched multiplication for small and medium matrix dimensions [10]. These methods substantially improve matrix-multiplication efficiency but retain coefficient-oriented CKKS representations that may require conversions when computations alternate between matrix operations and slot-domain SIMD operations.

The recently proposed GL scheme introduces a significantly different design point. GL provides native slot-encoded matrix arithmetic through a multivariate ring construction and a Gaussian-integer viewpoint, reducing homomorphic matrix multiplication to modular matrix multiplications on ciphertext coefficients [17]. Because graphics processing units (GPUs) are designed for high-throughput matrix multiplication, this structure motivates evaluating GL for matrix-intensive encrypted workloads, such as private inference, and investigating GPU acceleration in future work. The original GL paper notes that bootstrapping can in principle be applied to its ciphertext slices but does not specify a procedure specialized to the GL algebra [17]. Such a procedure is required to apply GL to computations that exceed its initial modulus budgets.

Designing bootstrapping for GL is substantially more challenging than directly applying a CKKS construction. The difficulty is algebraic, not merely implementation-related. Conventional CKKS bootstrapping constructions typically operate over a one-dimensional power-of-two cyclotomic ring, where slots-to-coefficients (StC) and coefficients-to-slots (CtS) transforms, automorphisms, and masking structures are well understood. In contrast, a GL ciphertext slice lives in a multivariate (X, W) ring containing the cyclotomic component $\Phi_p(W)$. In this work, we restrict p to be prime. This multivariate algebra changes the semantics of rotations and the action of sparse secret-key monomials during the decryption circuit. Consequently, standard CKKS bootstrapping techniques do not transfer directly. Any practical GL bootstrapping algorithm must therefore be derived from the slice algebra itself rather than obtained by a straightforward reuse of CKKS machinery.

Concurrent work by Crockett *et al.* [13] develops a classical EvalMod-based pipeline for GL. However, EvalMod inherently requires high multiplicative depth, restricting its feasibility to large RLWE ring dimensions (e.g., $\log_2 N \geq 15$). Because EvalMod depth reductions have largely saturated, our framework is the first to bypass these limitations in GL. By adapting the shallow decryption-in-the-exponent approach of SHIP [7], we unlock low-depth bootstrapping at smaller, more flexible ring dimensions. It begins with a grouped, depth-1 GL StC: the X -axis inversion uses native plaintext–ciphertext matrix multiplication, whereas the W -axis inversion uses a diagonal baby-step/giant-step (BSGS) transform, with all W -axis number-theoretic transforms (NTTs) evaluated by our optimized Rader algorithm. Each resulting Y -slice is decomposed into two Gaussian channels. Encrypted one-hot masks select the complete W -axis action, fine X -axis residue, and Gaussian phase from a public candidate family without revealing them. Only the remaining coarse X -axis displacement is applied through hidden X -only homomorphic multiplexer (HMux) rotations, after which a balanced product tree assembles the decryption phase in the exponent. Factoring the public candidates reduces the online plaintext encodings from the fully expanded candidate-family upper bound of $8\theta\varphi(p) + 2$ to $2\varphi(p) + 2$ per Y -slice, where θ controls the fine/coarse X -axis decomposition and $\varphi(p) = p - 1$. Our construction operates slice-wise in the (X, W) ring and preserves GL’s native matrix representation.

We derive and implement the construction for RLWE ring dimensions $\log_2 N \in \{13, 14, 15\}$. Under matched parameter sets providing 128 bits of security, our implementation achieves lower amortized per-slice latency than the matched CKKS baselines for both StC and full complex bootstrapping across the

evaluated configurations. These results demonstrate the feasibility of low-depth GL bootstrapping at small and moderate ring dimensions.

Our contributions are as follows:

- We develop a tailored StC algorithm for GL that performs the X -axis inversion using native GL plaintext–ciphertext matrix multiplication and the W -axis inversion using a BSGS linear transform.
- We derive a slice-wise SHIP-style half-bootstrap over the two Gaussian channels, show that hidden selection requires only X -axis HMux rotations, and prove the resulting construction correct.
- We reduce the number of online plaintext encodings and optimize the prime-length W -axis transform through a fused Rader NTT implementation. Together, the grouped GL StC, slice-wise half-bootstrap, and these optimizations yield faster amortized per-slice StC and full complex bootstrapping than the matched CKKS baselines.

Roadmap: The rest of the paper is organized as follows. In Section 2, we review the necessary background on the CKKS and GL schemes, alongside existing bootstrapping paradigms for approximate HE. In Section 3, we present our proposed low-depth bootstrapping architecture, detailing the grouped slots-to-coefficients (StC) transform that retains the Y -slice decomposition, followed by our slice-wise decryption-in-the-exponent half-bootstrap. Section 4 provides our performance evaluation and benchmarks, including comprehensive comparisons against CKKS baselines. Section 5 discusses related work on homomorphic matrix computations and positions our contribution relative to CKKS bootstrapping algorithms and other matrix-oriented FHE schemes. Finally, Section 6 concludes the paper.

2 Background

In this section, we briefly review the CKKS and GL schemes and summarize the main paradigms for bootstrapping CKKS ciphertexts. We then highlight why bootstrapping the GL scheme is substantially more challenging than directly reusing existing CKKS constructions.

2.1 Overview of the CKKS and GL Schemes

CKKS scheme. The CKKS scheme [11] is an RLWE-based fully homomorphic encryption scheme for *approximate* arithmetic over real and complex numbers. It is typically instantiated over the power-of-two cyclotomic ring

$$R_q^{\text{CKKS}} = \mathbb{Z}_q[X]/(X^N + 1),$$

where N is a power of two and q is the ciphertext modulus. Through the canonical embedding, a plaintext polynomial can encode $N/2$ complex values, enabling SIMD-style packed computation. To represent real or complex values with high precision, CKKS uses a scaling factor Δ , so that plaintext values are mapped to approximate ring elements.

A CKKS ciphertext has the usual RLWE form (b, a) , and decryption satisfies $b + as \approx m$, where s is the secret key and m is the encoded plaintext. Homomorphic addition is component-wise, while homomorphic multiplication produces a higher-degree ciphertext under the enlarged secret-key basis $(1, s, s^2)$. Therefore, multiplication must be followed by *relinearization*, which is a form of key switching that returns the ciphertext to the standard two-component form. In addition, multiplication increases the effective scale, so CKKS uses *rescaling* to reduce the modulus and bring the scale back under control. These operations make CKKS especially well suited for approximate numerical workloads such as neural network inference, where additions and multiplications dominate.

GL scheme. The Gentry–Lee (GL) scheme [17] is a recently proposed RLWE-based FHE construction designed for *native matrix arithmetic*. Instead of working in a univariate cyclotomic ring and treating matrices as packed vectors, GL uses a multivariate Gaussian-integer ring

$$R_q'^{\text{GL}} = \mathbb{Z}_q[I, X, Y, W]/\langle I^2 + 1, X^n - I, Y^n - I, \Phi_p(W) \rangle,$$

while the secret key is sampled from the smaller ring

$$R_q^{\text{GL}} = \mathbb{Z}_q[I, X, W] / \langle I^2 + 1, X^n - I, \Phi_p(W) \rangle.$$

Here, p is odd, Φ_p is the p -th cyclotomic polynomial, and, following GL's presentation, $\mathbb{Z}_p^*$ is assumed cyclic. Throughout this work, we restrict to prime p ; consequently, $\varphi(p) = p - 1$, and the primitive p -th roots are exactly the nontrivial p -th roots. With two Gaussian coordinates in the basis $\{1, I\}$, X -axis degree n , and W -axis degree $\varphi(p)$, each (X, W) slice has RLWE ring dimension $N = 2n\varphi(p)$. Throughout this paper, I denotes the formal Gaussian-ring generator, represented by the residue class satisfying $I^2 = -1$ in R_q^{GL} . In contrast, $i = \sqrt{-1} \in \mathbb{C}$ denotes the usual imaginary unit used by the canonical embedding, roots of unity, and complex approximation formulas. These symbols are not interchangeable. Under this construction, one plaintext polynomial encodes $\varphi(p)$ matrices of size $n \times n$ by evaluation at roots of unity:

$$m(i, \zeta_j^{(X)}, \zeta_k^{(Y)}, \eta_\ell) = M_{j,k}^{(\ell)},$$

where $\zeta_j^{(X)}$ and $\zeta_k^{(Y)}$ are the selected $4n$ -th roots satisfying $(\zeta_j^{(X)})^n = (\zeta_k^{(Y)})^n = i$, and η_ℓ ranges over the primitive p -th roots associated with the W -axis. This yields a native matrix encoding in which row and column structure are represented directly in the plaintext algebra. Under this restriction, the W -axis canonical transform is evaluated at the nontrivial p -th roots of unity and can be implemented using a length- p NTT. Rader's algorithm reduces this prime-length transform to a cyclic convolution of length $\varphi(p) = p - 1$ [27].

As a simple example, if $n = 2$ and $\varphi(p) = 2$ (which means $p = 3$), then a single GL plaintext can encode 2 separate 2×2 complex matrices at once. In this case, each Y -slice is an (X, W) -ciphertext with RLWE ring dimension $2n\varphi(p) = 8$, so one GL ciphertext can be viewed as two slice ciphertexts. The Y -slices are coefficient components of the message along the Y -axis; their joint Y -DFT reconstructs the encoded matrices.

Supported homomorphic operations in GL. The GL scheme naturally supports a broad set of matrix operations through its multivariate ring structure [17]. In particular:

- **Encrypted matrix multiplication:** it reduces plaintext–ciphertext matrix multiplication to two plaintext modular matrix multiplications, one on each ciphertext component. Ciphertext–ciphertext matrix multiplication uses four modular matrix multiplications between ciphertext components, followed by relinearization using two switching keys and rescaling.
- **Addition:** ciphertext addition is component-wise and directly corresponds to matrix addition in the encoded domain.
- **Hadamard multiplication:** element-wise matrix multiplication is realized by ordinary ciphertext multiplication, followed by relinearization and rescaling.
- **Conjugation:** complex conjugation of each encoded matrix is implemented by a ring automorphism together with key switching.
- **Row rotation:** cyclic row shifts inside each encrypted matrix are induced by automorphisms on the X -variable, followed by key switching.
- **Column rotation:** cyclic column shifts are induced by automorphisms on the Y -variable; since the secret key is independent of Y , this operation does not require key switching.
- **Inter-matrix rotation:** rotations across the batched matrices are induced by automorphisms $W \mapsto W^{g_W^j}$, where g_W generates $\mathbb{Z}_p^*$; these automorphisms permute the W -slots. This operation is distinct from multiplication by $W^v \bmod \Phi_p(W)$, which acts linearly on the W -coefficients and is evaluated in the public candidate table of our bootstrap.
- **Transpose:** matrix transposition is implemented by permuting the X - and Y -variables, followed by key switching.
- **Conjugate transpose:** Conjugate transposition is obtained through a combined automorphism on X , Y , and W , together with key switching.

These operations are important because they show that GL is not only a scheme for fast encrypted matrix multiplication, but a broader matrix-native homomorphic computation model. As noted in the GL paper, the resulting ciphertext structure can be combined with existing RLWE-based bootstrapping techniques by applying bootstrapping independently to each Y -slice [17].

Why GL is different from CKKS for bootstrapping. Although GL can be viewed as CKKS-like in the sense that it supports approximate arithmetic and is based on RLWE, its internal algebra is fundamentally different from standard CKKS. In CKKS, bootstrapping is designed around a one-dimensional power-of-two cyclotomic ring, where StC and CtS transforms, automorphisms, and masking structures are well understood. In contrast, a GL ciphertext slice lives in a multivariate (X, W) ring containing the cyclotomic component $\Phi_p(W)$, and the decryption circuit must account for Gaussian-integer coefficients and the interaction between the X - and W -axes. Consequently, standard CKKS bootstrapping methods do not transfer directly. This algebraic mismatch is precisely what makes GL bootstrapping both challenging and interesting.

2.2 Bootstrapping Paradigms for Approximate HE

Bootstrapping maps a low-level ciphertext to a high-level ciphertext that encrypts approximately the same message. Classical CKKS bootstrapping [6] follows

$$C_{\text{slot}} \xrightarrow{\text{StC}} C_{\text{coeff}} \xrightarrow{\text{ModRaise}_{q \rightarrow Q}} \widehat{C}_{\text{coeff}} \xrightarrow{\text{CtS}} \widehat{C}_{\text{slot}} \xrightarrow{\text{EvalMod}_q} C'_{\text{slot}}.$$

After modulus raising, decryption over Q yields $\mu + qk$, where μ is the desired approximate plaintext and k is an integer-valued overflow term. The CtS transform exposes these lifted coefficients as slots, and EvalMod_q evaluates a polynomial approximation to $z \mapsto z \bmod q$, so that $\text{EvalMod}_q(\mu + qk) \approx \mu$. This method is accurate but its polynomial modular reduction generally requires several multiplicative levels [20, 29].

SHIP bootstrapping. SHIP [7] instead evaluates decryption in the exponent. After StC, it switches the resulting level-zero coefficient ciphertext from the dense secret s_d to the sparse secret $s_{\text{sp}} = \sum_{e=0}^{h-1} \varepsilon_e X^{j_e}$, where j_e are its nonzero monomial positions, $\varepsilon_e \in \{-1, 1\}$, and h is its Hamming weight. Let q_0 denote the level-zero ciphertext modulus. For the resulting ciphertext (b, a) under s_{sp} , its modular decryption phase is

$$m = b + as_{\text{sp}} \pmod{q_0}.$$

If μ is the target approximate message and γ is SHIP's encoding gap, their coefficients satisfy

$$\langle m_k \rangle_{q_0} = \left\lfloor \frac{q_0 \mu_k}{\gamma} \right\rfloor$$

up to the ordinary CKKS encoding and encryption error. Let $\langle f_k \rangle_{q_0} \in [-q_0/2, q_0/2)$ denote the centered lift of the k -th coefficient of $f \in R_{q_0}^{\text{CKKS}}$. Following SHIP, with $\omega = \exp(2\pi i/q_0)$, define the two coefficient-wise root-of-unity vectors

$$\Omega_1(f) = (\omega^{\langle f_k \rangle_{q_0}})_{0 \leq k < N/2}, \quad \Omega_2(f) = (\omega^{\langle f_{k+N/2} \rangle_{q_0}})_{0 \leq k < N/2}.$$

SHIP encodes these vectors and their conjugates as plaintexts. Masked selection derives the required shifted factors, and the product tree computes

$$\Omega_1(m) = \Omega_1(b) \odot \prod_{e=0}^{h-1} \Omega_1(\varepsilon_e a X^{j_e}),$$

where each shifted polynomial is reduced in $R_{q_0}^{\text{CKKS}}$, $\odot$ denotes element-wise multiplication, and the product is evaluated element-wise. Thus, Ω_1 and Ω_2 apply roots of unity coefficient-wise after reduction and centered lifting modulo q_0 ; they are not exponentiation in the polynomial ring. Masked-column selection chooses the candidate for the fine interval offset, and blind rotation applies the remaining coarse interval offset for each j_e . The balanced product tree then assembles the resulting factors. Combining the result with its complex conjugate extracts $\frac{\gamma}{2\pi} \sin(2\pi \langle m_k \rangle_{q_0} / q_0)$, which is approximately μ_k over SHIP's valid input range.

Natural-order StC requirement. A depth-2 radix-2 Cooley–Tukey decomposition leaves the transformed values in bit-reversed coefficient order. This does not change the sparse-secret monomial positions j_e : the masks remain indexed by the actual polynomial coefficients, and SHIP still assembles $\omega^{b+a_{sp}}$ correctly coefficient-wise. However, SHIP’s half-bootstrap does not undo the bit reversal, so the refreshed values would appear in bit-reversed slot order [7]. We consequently use a depth-1 StC that merges the transform factors into one level and returns the coefficients in natural order. A bit-reversed StC would instead require an explicit output permutation.

Window and hybrid parameters. SHIP writes the bounded support interval using a radius w_{SHIP} , giving an interval of width $2w_{\text{SHIP}}$. Throughout this paper, w instead denotes the full implemented window width; thus, before any implementation-specific alignment, $w = 2w_{\text{SHIP}}$, and after alignment it corresponds to SHIP’s implemented interval length ℓ_0 . We therefore write the e -th public interval as $j_e \in [L_e, L_e + w)$. This w is unrelated to the GL ring variable W . The position offset is decomposed as

$$j_e - L_e = r_e + \theta t_e, \quad 0 \leq r_e < \theta, \quad 0 \leq t_e < \left\lceil \frac{w}{\theta} \right\rceil.$$

The masked column selects the fine interval offset r_e , while blind rotation selects the coarse interval index t_e . Hence, θ trades masked-column material against the blind-rotation range: increasing θ uses more column candidates but fewer blind-rotation steps.

Dense-to-sparse switching and the automorphisms used by SHIP employ hybrid key switching [18]. Abstractly, if $a = \sum_{\ell=0}^{d-1} a_\ell g_\ell$ is a gadget or RNS decomposition, an evaluation key contains encryptions $K_\ell = \text{Enc}_{s'}(Pg_\ell s)$, and

$$\text{KSwitch}_{s \rightarrow s'}(b, a) = (b, 0) + \text{ModDown}_P \left(\sum_{\ell=0}^{d-1} a_\ell K_\ell \right).$$

The auxiliary modulus P is removed after accumulation, and $d = d_{\text{num}}$ denotes the number of decomposition partitions. Thus, θ controls the masked-column/HMux split, whereas d_{num} controls RNS hybrid key switching. Our construction adopts this low-depth SHIP pipeline and adapts its masked selection and exponent computation to each GL ciphertext slice.

2.3 Threat Model

We consider an honest-but-curious outsourced-computation setting. A client provides ciphertexts and the corresponding public and evaluation keys to an untrusted server, which follows the prescribed homomorphic computation but may attempt to infer information from the material it observes.

Our evaluation material inherits SHIP’s sparse-secret and bounded-window assumptions [7]. For the e -th nonzero secret coefficient, the interval $[L_e, L_e + w)$ is public, whereas its exact position and value in $\{\pm 1, \pm I\}$ remain hidden by encrypted masks. Following SHIP, we account for this leakage using May’s attack on ternary LWE keys and choose the secret Hamming weight and window width to maintain at least 128 bits of estimated security [25]. As with standard FHE evaluation keys, we assume that the published RLWE encryptions of secret-dependent switching material and masks are secure under the usual evaluation-key assumption.

CKKS and GL achieve IND-CPA security by instantiating encryption as an RLWE encryption problem: a ciphertext is a noisy affine masking of the plaintext under a secret ring element, so distinguishing encryptions of two chosen messages is computationally hard assuming RLWE is hard over the underlying ring. In CKKS this ring is the standard cyclotomic RLWE ring, while in GL the multivariate Gaussian-integer construction is chosen so that it remains reducible to standard RLWE hardness over an appropriate cyclotomic ring [17].

Algorithm 1 Matrix-multiplication-based X -axis inversion

Input: A GL ciphertext C in representation DFT_{XYW}

Input: $P_X = \text{Encode}_{\Delta_X}^{\text{mat}}(E_X^*)$

Output: A ciphertext C in representation DFT_{YW}

1: $C \leftarrow \text{ConjTranspose}(C)$

2: $C \leftarrow \text{MatMul}(C, P_X)$

3: $C \leftarrow \text{ConjTranspose}(C)$

4: **return** C

3 Proposed Low-Depth Bootstrapping Architecture

3.1 Core Bootstrapping Construction for the GL Algebra

Our bootstrap first applies a grouped, depth-1 GL StC that inverts the X -axis through native plaintext-ciphertext matrix multiplication and the W -axis through a diagonal BSGS transform, while retaining the Y -slice decomposition. After switching to the sparse bootstrapping secret, it processes each Y -slice through two Gaussian channels. Encrypted masks select the complete W -axis action, fine X -axis residue, and Gaussian phase from ciphertext-dependent public candidates; an X -only HMux then applies the remaining coarse X -axis displacement. A balanced product tree assembles the decryption phase in the exponent, and SHIP's conjugate-and-add step recovers an approximation to each message channel. Finally, the channels are recombined with the formal element I , and the refreshed Y -slices are reassembled into the output GL ciphertext.

Grouped Slots-to-Coefficients (StC) Transform Write $m(X, Y, W) = \sum_{y=0}^{n-1} m_y(X, W)Y^y$. Since the GL secret is independent of Y , its coefficient slices m_y can be bootstrapped independently; only the X - and W -axis transforms must be inverted homomorphically. For $S \subseteq \{X, Y, W\}$, let DFT_S mean that precisely the axes in S remain in canonical evaluation form. Thus,

$$\text{DFT}_{XYW} \xrightarrow{\text{XInv}} \text{DFT}_{YW} \xrightarrow{\text{WInv, Rescale}} \text{DFT}_Y,$$

while an extracted (X, W) -slice is in $\text{DFT}_\emptyset$; DFT_{XW} denotes the refreshed slice. The two axis inversions act on independent axes and therefore commute. In the grouped schedule used in our experiments, both linear transforms are evaluated at the same ciphertext level and rescaling is delayed until the second transform finishes. This two-stage schedule consumes one modulus limb and one multiplicative level, taking a level-1 input to level 0. Alternatively, rescaling after each transform uses two limbs and two levels. Either schedule may evaluate X before W or W before X ; the implementation can select the faster order, and the last transform in the chosen order performs the delayed rescale. The algorithms below present the grouped X -then- W order.

X-axis homomorphic decoding. This step realizes the X -axis inversion using the forward X -axis DFT matrix independently for each fixed W -slot. Let

$$(E_X)_{j,x} = (\zeta_j^{(X)})^x.$$

For each W -slot we need the left action $E_X M$. GL provides a right plaintext-ciphertext matrix product, so we use $E_X M = (M^* E_X^*)^*$ with $P_X = \text{Encode}_{\Delta_X}^{\text{mat}}(E_X^*)$, repeated across the W -slots. The matrix encoder includes GL's trace normalization, and the final grouped rescale removes this stage's scale growth.

W-axis homomorphic decoding. This step realizes the W -axis inversion using the forward W -axis DFT matrix across the $\varphi(p)$ batched-matrix slots. Let g_W generate $\mathbb{Z}_p^*$, order the slots as $\eta_r = e^{2\pi i g_W^r/p}$, and define

$$\begin{aligned} (U_W)_{r,w} &= \eta_r^w, & (\rho_W^j z)_r &= z_{r+j \bmod \varphi(p)}, \\ d_t[r] &= (U_W)_{r,r+t \bmod \varphi(p)}, & U_W z &= \sum_{t=0}^{\varphi(p)-1} d_t \odot \rho_W^t(z). \end{aligned}$$

Algorithm 2 BSGS-based W -axis inversion

Input: A GL ciphertext C in representation DFT_{YW} **Input:** Precomputed diagonal plaintexts $\{D_t\}_{t=0}^{\varphi(p)-1}$ **Input:** Baby-step size m **Output:** A ciphertext A in representation DFT_Y

```
1:  $G \leftarrow \lceil \frac{\varphi(p)}{m} \rceil$ 
2: for  $j = 0$  to  $m - 1$  do
3:    $C_j \leftarrow \rho_W^j(C)$ 
4:  $A \leftarrow 0$ 
5: for  $b = 0$  to  $G - 1$  do
6:    $T_b \leftarrow 0$ 
7:   for each valid  $t \in \{bm, \dots, \min(\varphi(p) - 1, bm + m - 1)\}$  do
8:      $j \leftarrow t - bm$ 
9:      $T_b \leftarrow T_b + C_j \odot \rho_W^{-bm}(D_t)$ 
10:   $A \leftarrow A + \rho_W^{bm}(T_b)$ 
11:  $A \leftarrow \text{Rescale}(A)$ 
12: return  $A$ 
```

Here U_W is the forward W -axis DFT matrix, $D_t = \text{Encode}_{\Delta_W}(d_t)$, and $\odot$ is element-wise plaintext-ciphertext multiplication. On ciphertexts, ρ_W^j is the automorphism $W \mapsto W^{g_W^j}$ followed by key switching with a small W -rotation key; on plaintexts it is a free permutation. It is distinct from coefficient multiplication by $W^v \bmod \Phi_p(W)$.

Algorithm 2 uses $m - 1$ baby and $\lceil \varphi(p)/m \rceil - 1$ giant rotations, with hoisted decomposition reused within each group.

Slice-Wise Decryption-in-the-Exponent (Half-Bootstrapping) The half-bootstrap accepts a level-zero ciphertext in DFT_Y , switches it from the dense to the sparse bootstrapping secret, and extracts its n slices in $\text{DFT}_\emptyset$. For one slice (B, A) , write $A = A_0 + IA_1$, $B = B_0 + IB_1$, and $s_{\text{sp}} = s_0 + Is_1$. For the level-zero modulus q_0 , its modular phase and Gaussian channels are

$$\begin{aligned} m &= B + As_{\text{sp}} = m_0 + Im_1 \pmod{q_0}, \\ m_0 &= B_0 + A_0s_0 - A_1s_1, \quad m_1 = B_1 + A_0s_1 + A_1s_0. \end{aligned} \tag{1}$$

For $Z = Z_0 + IZ_1$, let $[Z]_\alpha = Z_\alpha$. At logical coordinate k , SHIP's gap- γ encoding relates the centered phase to the target approximate message $\mu = \mu_0 + I\mu_1$ by

$$m_{\alpha,k} := \langle [m]_{\alpha,k} \rangle_{q_0} = \lfloor q_0 \mu_{\alpha,k} / \gamma \rfloor, \quad \alpha \in \{0, 1\}. \tag{2}$$

Encoding and homomorphic errors are treated separately. If $\rho = \varepsilon I^\chi$, then

$$[\rho Z]_\alpha = \sigma_{\alpha,\rho} Z_{\beta_{\alpha,\rho}}, \quad \beta_{\alpha,\rho} = \alpha \oplus \chi, \quad \sigma_{\alpha,\rho} = \varepsilon(-1)^{(1-\alpha)\chi}. \tag{3}$$

Thus α is the output channel, while β and σ select the source component and sign. The two channels are recombined with the formal element I after both SHIP branches.

Sparse support and target. In W -major order, sparse-secret term e has

$$f_e = v_e(2n) + 2u_e + \chi_e, \quad 0 \leq u_e < n, \quad 0 \leq v_e < \varphi(p), \quad \chi_e \in \{0, 1\}. \tag{4}$$

Its sign is $\varepsilon_e \in \{\pm 1\}$. It represents

$$c_e X^{u_e} W^{v_e}, \quad c_e = \varepsilon_e I^{\chi_e} \in \{\pm 1, \pm I\}.$$

For a candidate f , define $v(f) = \lfloor f/(2n) \rfloor$, $u(f) = \lfloor (f \bmod 2n)/2 \rfloor$, and $\chi(f) = f \bmod 2$. SHIP's bounded-window assumption gives the public full-width interval

$$f_e \in \mathcal{I}_e = [L_e, L_e + w) \subseteq [0, 2n\varphi(p)), \quad (5)$$

where L_e is its public start and w its width. This W -major interval includes the Gaussian component bit and is not an interval over $\varphi(p)u + v$.

With $s_{\text{sp}} = \sum_{e=0}^{h-1} c_e X^{u_e} W^{v_e}$, each channel targets

$$\begin{aligned} [m]_\alpha &= [B]_\alpha + \sum_{e=0}^{h-1} [Ac_e X^{u_e} W^{v_e}]_\alpha \pmod{q_0}, \\ \omega[m]_\alpha &= \omega[B]_\alpha \prod_{e=0}^{h-1} \omega[Ac_e X^{u_e} W^{v_e}]_\alpha. \end{aligned} \quad (6)$$

Here $\omega = e^{2\pi i/q_0}$. Each channel therefore needs one B -factor and one secretly selected factor per sparse term; a balanced product tree combines them. Two channels are required because $c_e \in \{\pm 1, \pm I\}$ mixes the Gaussian components.

Masked column and coarse X -axis HMux. For the hybrid decomposition parameter $\theta \mid n$, write

$$u_e = u_{0,e} + U_e, \quad u_{0,e} = u_e \bmod \theta, \quad U_e = \theta \lfloor u_e / \theta \rfloor. \quad (7)$$

SHIP's displayed hybrid algorithm assigns the coarse component to the column method and the fine component to HMux, but its implementation uses the reverse ordering [7]; we follow the implementation, selecting the fine X -axis residue $u_{0,e}$ by masked column and applying the coarse X -axis displacement U_e by HMux. For $f \in \mathcal{I}_e$ and $\varepsilon \in \{\pm 1\}$, set

$$u_0(f) = u(f) \bmod \theta, \quad U(f) = u(f) - u_0(f), \quad \rho_{f,\varepsilon}(r_0) = \varepsilon I^{\chi(f)+1[r_0 < u_0(f)]+1[r_0 \geq n-U(f)]}.$$

The row-wise reachable candidates and their public union are

$$\begin{aligned} \mathcal{C}_e(r_0) &= \{(u_0(f), v(f), \rho_{f,\varepsilon}(r_0)) : f \in \mathcal{I}_e, \varepsilon \in \{\pm 1\}\}, \\ \mathcal{C}_e &= \bigcup_{r_0=0}^{n-1} \mathcal{C}_e(r_0). \end{aligned} \quad (8)$$

The key stores one encrypted row-wise mask per $c \in \mathcal{C}_e$, hence $\sum_e |\mathcal{C}_e|$ masks in total.

The window also determines the possible coarse X -axis indices

$$\mathcal{H}_e = \{\lfloor u(f)/\theta \rfloor \bmod (n/\theta) : f \in \mathcal{I}_e\} \subseteq \{(j_{\text{base},e} + \ell) \bmod (n/\theta) : 0 \leq \ell < M_e\},$$

where the right side is the smallest cyclic interval containing $\mathcal{H}_e$. Cyclicity is necessary because a W -major window can cross a W -block boundary and wrap around n/θ . The secret residual is

$$j_{\text{res},e} = \left(\frac{U_e}{\theta} - j_{\text{base},e} \right) \bmod \frac{n}{\theta}, \quad 0 \leq j_{\text{res},e} < M_e. \quad (9)$$

For radix $\mathcal{B} \geq 2$, the key hk_e for sparse term e encrypts the base- $\mathcal{B}$ digits of $j_{\text{res},e}$; $\lceil \log_{\mathcal{B}} M_e \rceil$ HMux stages, including the public base shift, compute

$$\text{HMux}_e^X(C; \text{hk}_e) = \text{Rot}_{\theta(j_{\text{base},e} + j_{\text{res},e})}^X(C) = \text{Rot}_{U_e}^X(C), \quad (10)$$

with indices modulo n , without revealing U_e or $j_{\text{res},e}$. At each stage, the evaluator forms the possible X -rotated branches for one radix digit and selects one using encrypted digit selectors. The selected displacements

accumulate across the stages, revealing neither the digits nor the combined rotation index. Thus the masked column selects the fine X -axis residue, complete W exponent, and row-dependent Gaussian phase, while only the coarse X -axis displacement requires hidden rotation. We reserve q, q_0 for moduli; r is an output row and r_0 its row before this rotation.

Let $r_0 = (r - U_e) \bmod n$. Write $\tilde{A}$ and $\tilde{B}_\alpha$ for the centered coefficient arrays of A and $[B]_\alpha$ modulo q_0 , and extend $\tilde{A}_{x,p-1}^* = 0$. Under our prime- p restriction, $\Phi_p(W) = 1 + \dots + W^{p-1} = 0$ implies $W^p = 1$ and $W^{p-1} = -(1 + \dots + W^{p-2})$, shifting the extended vector by v and eliminating its W^{p-1} coefficient gives

$$\mathcal{A}_{u_0,v}(r_0, t) = \tilde{A}_{r_0 - u_0 \bmod n, (t-v) \bmod p}^* - \tilde{A}_{r_0 - u_0 \bmod n, (p-1-v) \bmod p}^*. \quad (11)$$

This public map is coefficient multiplication by $W^v \bmod \Phi_p(W)$, not a cyclic rotation of the stored coefficients. The required pre-HMux phase is

$$\rho_e(r_0) := \rho_{f_e, \varepsilon_e}(r_0) = c_e I^{\mathbf{1}[r_0 < u_{0,e}] + \mathbf{1}[r_0 \geq n - U_e]}. \quad (12)$$

The indicators encode the two $X^n = I$ wraps. Their intervals are disjoint because $u_{0,e} + U_e < n$, so the exponent is zero or one. The public channel candidate is

$$\Omega_{u_0,v,\rho}^{(\alpha)}(r_0, t) = \omega^{[\rho \mathcal{A}_{u_0,v}(r_0, t)]_\alpha}. \quad (13)$$

All exponents are centered modulo the level-zero modulus q_0 . Candidate $c = (u_0, v, \rho) \in \mathcal{C}_e$ has row-wise mask

$$M_{e;u_0,v,\rho}(r_0, t) = \mathbf{1}[u_0 = u_{0,e}] \mathbf{1}[v = v_e] \mathbf{1}[\rho = \rho_e(r_0)]. \quad (14)$$

It is constant in t but selected independently per row because the wrap phase can vary with r_0 . The key stores $M_{e,c} = \text{Enc}_{s_d}(M_{e;u_0,v,\rho})$ under the dense output secret s_d . Writing $\Omega_c^{(\alpha)} = \Omega_{u_0,v,\rho}^{(\alpha)}$, the masked column is

$$\text{MC}_e^{(\alpha)} = \text{ModDown}_P \left(\sum_{c \in \mathcal{C}_e} M_{e,c} \odot \text{Encode}(\Omega_c^{(\alpha)}) \right). \quad (15)$$

Let $Q_\ell = \prod_{j=0}^\ell q_j$ be the computational modulus available to the half-bootstrap and let P be the auxiliary key-switching modulus. The masks are encrypted over PQ_ℓ , with their selectors at scale P , whereas the online candidate plaintexts are encoded at the product-tree scale Δ . Thus, ModDown_P in Equation (15) removes only P and returns $\text{MC}_e^{(\alpha)}$ modulo Q_ℓ at scale Δ . The level-zero components $A, B \bmod q_0$ are used only to generate the public exponent tables; the encrypted masks thereby produce factors modulo Q_ℓ . The HMux is an auxiliary-modulus key switch, so it preserves this modulus and scale and consumes no multiplicative level. Exactly one candidate contributes in each row, without revealing the selected candidate or sparse-secret position. The result is already encrypted under s_d ; both channels reuse the same masks and differ only in their public, ciphertext-dependent $\Omega^{(\alpha)}$ plaintexts. Consequently, no encrypted W -axis selection is required.

One sparse-secret term. For $c_e X^{u_e} W^{v_e}$, first split $u_e = u_{0,e} + U_e$. The public map $\mathcal{A}_{u_0,v}$ applies the complete $W^{v_e} \bmod \Phi_p(W)$ action and the fine X -axis residue $u_{0,e}$, while $\rho_e(r_0)$ accounts for c_e and the wraps induced by $X^n = I$. The encrypted row masks select, at row r_0 , the candidate $(u_{0,e}, v_e, \rho_e(r_0))$, and the X -only HMux applies the remaining coarse X -axis displacement U_e . Thus, for each channel α ,

$$D_e^{(\alpha)} \longmapsto \omega^{[Ac_e X^{u_e} W^{v_e}]_\alpha},$$

which is the factor contributed by term e to the product tree.

Plaintext encoding. $\text{Encode}(z)$ applies the inverse X, W canonical transforms, scales, and rounds a logical table z , yielding a plaintext whose logical state is DFT_{XW} . The public, ciphertext-dependent B -factor and Ω tables are generated online from A, B ; only the encrypted masks and HMux keys are pre-generated. For dense-key $C \in \text{DFT}_{XW}$, $\text{Conj}_{XW}(C)$ is the homomorphic conjugation $(I, X, W) \mapsto (-I, X^{-1}, W^{-1})$, including key switching, and therefore conjugates every logical value.

Algorithm 3 Bootstrapping in our GL bootstrapping framework

Input: $h + 1$ is a power of two; keys $\text{ks}_{\text{d} \rightarrow \text{sp}}, \{\text{M}_{e,c}\}, \{\text{hk}_e\}$

- 1: **function** HALFBOTSTRAP(C_{coeff})
- 2: $C_{\text{sp}} \leftarrow \text{KeySwitch}(C_{\text{coeff}}; \text{ks}_{\text{d} \rightarrow \text{sp}})$
- 3: Extract $\{C_y = (B, A)\}_{y=0}^{n-1}$ from C_{sp}
- 4: **for** each slice $C_y = (B, A)$ **do**
- 5: Decompose $A = A_0 + IA_1$ and $B = B_0 + IB_1$
- 6: **for** $\alpha \in \{0, 1\}$ **do**
- 7: $\text{PT}_B^{(\alpha)} \leftarrow \text{Encode}(-i\gamma/(4\pi) \cdot \omega^{\tilde{B}\alpha})$
- 8: Generate $\Omega_{u_0,v,\rho}^{(\alpha)}$ from A using Equation (13)
- 9: **for** each sparse term e and channel α **do**
- 10: $\text{MC}_e^{(\alpha)} \leftarrow \text{MaskedColumn}_e(\Omega^{(\alpha)}; \{\text{M}_{e,c}\}_c); \quad D_e^{(\alpha)} \leftarrow \text{HMux}_e^X(\text{MC}_e^{(\alpha)}; \text{hk}_e)$
- 11: **for** $\alpha \in \{0, 1\}$ **do**
- 12: $P_\alpha \leftarrow \text{PT}_B^{(\alpha)} \cdot \prod_e D_e^{(\alpha)}; \quad C'_{y,\alpha} \leftarrow P_\alpha + \text{Conj}_{XW}(P_\alpha)$
- 13: $C'_y \leftarrow C'_{y,0} + I C'_{y,1}$
- 14: **return** $\text{Reassembly}_Y(\{C'_y\}) \in \text{DFT}_{XYW}$
- 15: **function** FULLBOTSTRAP(C)
- 16: $C_{\text{coeff}} \leftarrow \text{StC}_{\text{GL}}(C)$
- 17: **return** HALFBOTSTRAP(C_{coeff})

Algorithm 3 exposes both the half- and full-bootstrap interfaces. For each slice, the evaluator generates public candidate plaintexts from A and one B -dependent base plaintext per channel. For each sparse-secret term e , the masked column selects the fine X -axis residue, complete W action, and Gaussian phase, while the term-specific HMux applies the remaining coarse X -axis displacement. A balanced tree multiplies the resulting h factors with the scale- Δ B -factor, relinearizing each ciphertext-ciphertext product and rescaling after every multiplication layer, to compute $(\gamma/(4\pi i))\omega^{[m]\alpha}$. For $h = 31$, this tree has depth $\log_2(h + 1) = 5$ and maps Q_ℓ to $Q_{\ell-5}$; together with depth-1 GL StC, the complete bootstrap uses six levels. Homomorphic conjugation extracts the scaled sine approximation, and the two channels are recombined using the formal element I ; they run in parallel and do not add depth.

Output representation. Each channel leaves the slice in DFT_{XW} . Multiplication of the second channel by formal I maps $(z_0, z_1) \mapsto (-z_1, z_0)$, after which the channels are added. For a fixed (X, W) coordinate, let $(F_Y)_{j,y} = (\zeta_j^{(Y)})^y$. GL stores $c = F_Y^{-1}M = n^{-1}F_Y^*M$; reassembly places refreshed c_y at Y^y , so its logical DFT_{XYW} value is

$$\widehat{M}_j = \sum_{y=0}^{n-1} \widehat{c}_y (\zeta_j^{(Y)})^y. \quad (16)$$

No homomorphic Y -transform is performed: the refreshed slices are placed directly at their corresponding Y -coefficients, and their joint Y -DFT gives Equation (16).

Formal Correctness of the Half-Bootstrap

Statement. Let $m = B + As_{\text{sp}} \pmod{q_0}$ and let μ satisfy Equation (2). Within SHIP's approximation range [7], Algorithm 3 returns a dense-key encryption of $\widehat{\mu} = \widehat{\mu}_0 + I\widehat{\mu}_1$, where

$$\widehat{\mu}_{\alpha,k} = \frac{\gamma}{2\pi} \sin\left(\frac{2\pi m_{\alpha,k}}{q_0}\right) + \varepsilon_{\alpha,k}^{\text{HE}} \approx \mu_{\alpha,k}. \quad (17)$$

Here ε^{HE} collects encoding, key-switching, rescaling, multiplication, and ciphertext-noise errors; the remaining deviation is the rounding and sine-approximation error.

Proof. For channel α , Equations (11) and (12) show that, at row r_0 , the row-wise mask selects the candidate $(u_{0,e}, v_e, \rho_e(r_0))$. Equation (10) then applies U_e , so at every logical coordinate

$$D_e^{(\alpha)} \mapsto \omega^{[Ac_e X^{u_e} W^{v_e}]_\alpha},$$

up to homomorphic error. Multiplying these factors with the base plaintext $-(i\gamma/(4\pi))\omega^{[B]_\alpha}$ gives

$$P_\alpha = \frac{\gamma}{4\pi i} \omega^{[m]_\alpha}.$$

At coordinate k , homomorphic conjugation yields

$$P_{\alpha,k} = \frac{\gamma}{4\pi i} \exp\left(\frac{2\pi i m_{\alpha,k}}{q_0}\right), \quad \overline{P_{\alpha,k}} = -\frac{\gamma}{4\pi i} \exp\left(-\frac{2\pi i m_{\alpha,k}}{q_0}\right).$$

Therefore

$$P_{\alpha,k} + \overline{P_{\alpha,k}} = \frac{\gamma}{2\pi} \sin\left(\frac{2\pi m_{\alpha,k}}{q_0}\right), \quad \frac{2\pi m_{\alpha,k}}{q_0} = \frac{2\pi \mu_{\alpha,k}}{\gamma} + \delta_{\alpha,k}, \quad |\delta_{\alpha,k}| \leq \frac{\pi}{q_0}.$$

The near-linear sine range gives Equation (17). Applying this argument to both channels and returning $C'_{y,0} + IC'_{y,1}$ gives $\hat{\mu} \approx \mu$ in DFT_{XW} . Equation (16) proves the reassembled half-bootstrap; composing with GL StC proves the full bootstrap. $\square$

3.2 Algorithmic and Arithmetic Optimizations

Streamed and efficient plaintext generation. Real-valued CKKS SHIP uses five ciphertext-dependent plaintexts: one B -dependent and four A -dependent [7]. A complex bootstrap needs six distinct plaintexts—two B -dependent and four shared A -dependent. For GL, materializing both channels, θ fine X -axis residues, $\varphi(p)$ W exponents, and four phases gives the fully expanded upper bound $8\theta\varphi(p) + 2$, although public windows may reach fewer candidates. We instead factor Equation (11). For $\beta \in \{0, 1\}$, define

$$F_{\beta,y}^+(r_0) = \omega^{[\tilde{A}_{r_0,y}^*]_\beta}, \quad F_{\beta,y}^-(r_0) = \omega^{-[\tilde{A}_{r_0,y}^*]_\beta},$$

and extend both tables by $F_{\beta,p-1}^+ = F_{\beta,p-1}^- = 1$, and let $\Pi_{\beta,v}^+$ denote the stored positive base for component β and $u_0 = 0$:

$$\Pi_{\beta,v}^+(r_0, t) = F_{\beta,(t-v) \bmod p}^+(r_0) \cdot F_{\beta,(p-1-v) \bmod p}^-(r_0).$$

We store these $2\varphi(p)$ bases and two B -plaintexts. For channel α and phase ρ , Equation (3) selects $\Pi_{\beta_{\alpha,\rho},v}^+$ or its conjugate according to $\sigma_{\alpha,\rho}$, then applies the fine-residue X -index mapping to produce $\Omega_{u_0,v,\rho}^{(\alpha)}$. These choices only reinterpret the stored plaintext entries; they require no ciphertext automorphism, additional encoding, temporary plaintext, or key switch.

Plaintexts are generated and consumed independently for each Y -slice. Both Gaussian channels reuse the compact $2\varphi(p)$ base pool through read-only views, avoiding the expanded candidate family and reducing encoding work and peak memory.

Fused Rader NTT. For the prime- p instances used here, $d = \varphi(p) = p - 1$ is a power of two. Rader's algorithm [27] reindexes the nonzero indices by powers of a generator $g \in \mathbb{Z}_p^*$ and reduces the length- p transform to a length- d cyclic convolution. Fix a primitive p -th root $\eta \in \mathbb{Z}_q$, write $x(W) = \sum_{j=0}^{p-2} x_j W^j$, and set $x_{p-1} = 0$. For $a_t = x_{g^t}$ and $b_t^+ = \eta^{g^{-t}}$,

$$Y_m = x(\eta^{g^m}) = x_0 + (a * b^+)_{-m \bmod d}, \quad (a * b)_r = \sum_{t=0}^{d-1} a_t b_{r-t \bmod d}.$$

Let $\mathcal{F}_d$ and $\mathcal{G}_d$ be the forward and unnormalized inverse radix-two NTTs over $\mathbb{Z}_q$, so that $\mathcal{G}_d(\mathcal{F}_d(z)) = dz$. Then

$$a * b^+ = \mathcal{G}_d(\mathcal{F}_d(a) \odot K^+), \quad K^+ = d^{-1}\mathcal{F}_d(b^+).$$

Thus, the online computation is one generator permutation, one forward NTT, one pointwise product, one unnormalized inverse NTT, and output reconstruction. The inverse uses $b_t^- = \eta^{-g^{-t}}$ analogously, with $K^- = (pd)^{-1}\mathcal{F}_d(b^-)$; the factor p^{-1} supplies the inverse length- p normalization. All inverses in these expressions are modulo q .

Our realization precomputes the NTT-domain kernels $K^\pm$, their Shoup forms, the generator permutations, and the reconstruction constants for each (p, q) . It fuses the pointwise product into the inverse radix-two pipeline and reuses $\mathcal{F}_d(a)_0 = \sum_t a_t$. Since $\sum_t b_t^\pm = -1$, this bin requires multiplication by $-d^{-1}$ (after the inverse path's p^{-1} scaling). Writing $z = hd + r$ and $q = \mu d + 1$, where $0 \leq r < d$, we compute it exactly as $-zd^{-1} \equiv r\mu - h \pmod{q}$. Inserting x_0 in the same frequency bin broadcasts it through $\mathcal{G}_d$, and the inverse's missing evaluation at 1 is recovered from the skipped convolution entry instead of a separate weighted sum. Lazy Shoup butterflies reduce only when required, unity twiddles use add-subtract butterflies by skipping the modular multiplication, and each thread reuses one scratch buffer.

4 Performance Evaluation and Benchmarks

All experiments were conducted on an Ubuntu workstation equipped with an AMD 7970X CPU with 32 cores and 160 GB of RAM. All C++ code was compiled with GCC 13.3.0. All benchmarks were executed using a single CPU thread to accurately measure algorithmic improvements independent of multi-threading overhead and OS scheduler variance.

4.1 Implementation Methodology

We implemented both CKKS and GL from scratch in C++. We use OpenBLAS [33] as the backend for dense matrix multiplication. Both implementations use hybrid key switching [18] and the same rescaling, ModUp, and ModDown algorithms. The core NTT butterflies and radix algorithms are also implemented from scratch. Because the original implementation is not publicly available, we implemented CKKS SHIP bootstrapping [7] from scratch and compare our measurements with its reported results. Notably, the GL scheme has previously only been implemented in the proprietary Desilo library [14]. Although this library allows users to execute FHE operations via a Python interface, its closed-source nature obscures the internal cryptographic mechanisms. Therefore, evaluating and extending the scheme necessitated developing a custom C++ implementation.

4.2 Cryptographic Parameter Selection

Table 1 summarizes the secure GL and CKKS SHIP parameter sets utilized in our experiments. Here, Q denotes the ciphertext-modulus chain and P denotes the auxiliary modulus used for hybrid key switching. The StC bits column gives the bit size of the single grouped modulus limb that contains both transform stages. The mult. depth column gives the multiplicative levels available outside bootstrapping. It excludes the six levels consumed by the complete bootstrap: five for the balanced product tree and one for the grouped StC, as described in Section 3.1.

All parameter sets target 128 bits of security and use a dense uniform-ternary main secret. For the sparse secret used during bootstrapping, we follow SHIP [7]: we use its bounded-window assumption and sparse-secret Hamming weight $h = 31$, which SHIP estimates to provide 128 bits of security under its bounded-window analysis. We report w as the complete implemented window width, rather than SHIP's window radius. The recent homomorphic encryption standard [3] gives $\log_2(PQ)$ limits of 214, 430, and 868 bits for $N = 2^{13}, 2^{14}, 2^{15}$, respectively. All utilized GL and matched CKKS parameter sets remain within their corresponding limits.

Table 1. Secure parameter sets utilized for our GL bootstrapping framework and CKKS SHIP. The rightmost column gives the 128-bit $\log_2(PQ)$ limit for a uniform-ternary secret from the recent homomorphic encryption standard [3].

Scheme	Parameter set	Ring structure			Modulus size (bits)			dnum	γ	StC bits	Mult. depth	Max. $\log(PQ)$
		N	n	$\varphi(p)$	$\log Q$	$\log P$	$\log(PQ)$					
GL	n256p17	2^{13}	256	16	180	34	214	8	2^5	26	1	214
GL	n512p17	2^{14}	512	16	338	92	430	4	2^{11}	37	1	430
GL	n1024p17	2^{15}	1024	16	641	220	861	4	2^{11}	39	8	868
CKKS	LL13-secure	2^{13}	–	–	186	28	214	8	2^5	20	1	214
CKKS	LL14-secure	2^{14}	–	–	338	92	430	4	2^{11}	37	1	430
CKKS	HT15-secure	2^{15}	–	–	641	220	861	4	2^{11}	39	8	868

Table 2. Parameter sets and evaluation results for real-number bootstrapping reported in [7], and our implementation. The $\log(PQ)$, SHIP latency, and SHIP precision values are copied directly from [7] because its implementation is not open source. Precision is reported in bits as $-\log_2(e_{\max})$, where $e_{\max}$ is the maximum real-slot approximation error.

Parameter set	N	$\log(PQ)$	Implementation	Precision (bits)	Latency (s)	$T_{\text{SHIP}}/T_{\text{ours}}$
LL13	2^{13}	218	SHIP	4.45	3.02	$1.10\times$
			Ours	5.18	2.75	
LL14	2^{14}	437	SHIP	16.9	4.91	$1.03\times$
			Ours	17.22	4.77	
HT14	2^{14}	438	SHIP	4.55	13.2	$0.94\times$
			Ours	6.00	14.10	
HT15	2^{15}	881	SHIP	18.1	29.6	$1.08\times$
			Ours	18.64	27.41	

4.3 Baseline Validation Against CKKS SHIP

We compare our CKKS SHIP implementation against the reported results of SHIP in [7]. We report the published SHIP parameter sets and evaluation numbers alongside our own measurements in order to assess how closely our implementation matches their reported performance. This comparison uses the original SHIP parameter sets to match the published experiment; the security-adjusted parameter sets used elsewhere in this work are reported in Table 1. For consistency, all latency values in Table 2 are expressed in seconds. The final column reports $T_{\text{SHIP}}/T_{\text{ours}}$; a value greater than one indicates that our implementation is faster, whereas a value below one indicates that the reported SHIP [7] implementation is faster.

As shown in Table 2, our implementation is competitive with the reported SHIP results across all parameter sets. The measured latencies differ by at most approximately 9%: our implementation is faster for LL13, LL14, and HT15, and slower for HT14. These results should be interpreted as an implementation-level comparison because the measurements use different hardware and the reported SHIP implementation uses hand-optimized AVX2 kernels.

For all complex-valued experiments in the following subsections, the real and imaginary components of each input are sampled independently and uniformly from $[-1, 1]$.

4.4 Comparative Evaluation of the StC Transform

A depth-2 Cooley–Tukey StC leaves the transformed values in bit-reversed coefficient order. SHIP still evaluates $b+as$ and its root-of-unity factors correctly coefficient-wise because the sparse-secret masks remain

Table 3. Depth-1, two-stage StC comparison using the secure configurations in Table 1. The matched RLWE dimensions satisfy $N = 2n\varphi(p)$ for the GL configuration, and the GL latency is amortized over n slices.

CKKS/GL parameter sets	N	n	p	$\varphi(p)$	Scheme	Latency (s)	$T_{\text{CKKS}}/T_{\text{GL}}$
LL13-secure n256p17	2^{13}	256	17	16	CKKS GL	0.0398 0.0135	$2.95\times$
LL14-secure n512p17	2^{14}	512	17	16	CKKS GL	0.0902 0.0354	$2.55\times$
HT15-secure n1024p17	2^{15}	1024	17	16	CKKS GL	0.3343 0.1071	$3.12\times$

indexed by the actual polynomial coefficients. Its half-bootstrap does not, however, undo the bit reversal, so the refreshed ciphertext would contain the message in bit-reversed slot order [7]. We therefore group the two CKKS StC stages within one level and emit the coefficients in natural order, avoiding a separate output permutation, as detailed in Section 2.2.

Table 3 shows that the GL StC is $2.55\times$ – $3.12\times$ faster than the matched CKKS transform. The advantage persists as the absolute transform latency increases with the ring dimension, and the largest relative improvement occurs at $N = 2^{15}$. This behavior is consistent with the different decomposition of the linear transform: GL evaluates the X -axis inversion through native plaintext–ciphertext matrix multiplication and confines the diagonal BSGS transform to the W -axis, whereas CKKS applies rotation-based linear transformations over its one-dimensional packed vector. Thus, the larger parameter sets preserve the structural advantage of the GL transform, although the precise speedup depends on the transform schedule and key-switching parameters.

4.5 Comparative Evaluation of Full Bootstrapping Throughput

We evaluate complex GL and CKKS bootstrapping under the secure parameter sets in Table 1. The matched CKKS baseline uses our complex SHIP implementation and the same implementation framework. The GL latency is amortized over the n slices contained in one GL ciphertext. As in Table 1, the multiplicative-depth column reports the levels available outside the six levels consumed by bootstrapping.

We observe a modest $1.09\times$ – $1.16\times$ speedup over the matched CKKS configurations. This gain is enabled by our optimized Rader kernels, which reduce the arithmetic overhead of the prime-length W -axis NTTs and make them competitive with the one-dimensional radix-2 NTTs used by CKKS, together with the considerably faster GL StC reported in Section 4.4. Moreover, GL exhibits lower full-output precision because the final Y -DFT reconstruction accumulates approximation errors from the independently bootstrapped Y -slices, which can increase the maximum absolute error in the reconstructed matrix.

4.6 Memory Footprint and Evaluation-Key Analysis

The dominant memory terms are the StC keys and the SHIP masked-column and HMux keys; the latter two together form the hybrid blind-rotation key. The logical HMux count depends on the coarse X -axis rotation range remaining after the θ -split and on the decomposition radix $\mathcal{B} = 4$. In contrast, dnum does not change this count; it determines the number of hybrid RNS partitions stored and accumulated within each HMux branch and therefore affects key memory. For CKKS, we use the value of θ optimized in the SHIP paper. For GL, we tune θ subject to the algebraic constraint $\theta \mid n$. Its masked-column count is $\sum_e |\mathcal{C}_e|$, rather than CKKS’s fixed $4\theta h$ count in these configurations: a W -major public window can cross a W -block boundary and introduce candidates for multiple W exponents, so $|\mathcal{C}_e|$ varies with e . Consequently, GL can require more masks even with a smaller θ , as in LL13 (848 versus 744).

Table 4. Complex-bootstrapping measurements. Precision is $-\log_2(e_{\max})$, where $e_{\max}$ is the maximum error measured in the corresponding experiment. GL latency is amortized over n slices. The window width w is shared by the matched CKKS and GL configurations; θ is scheme-specific. Each GL configuration uses $p = 17$ and $\varphi(p) = 16$. The latency ratio is $T_{\text{CKKS}}/T_{\text{GL}}$.

CKKS/GL parameter sets	N	n	w	Scheme	θ	Mult. depth	Precision (bits)	Latency (s)	Latency ratio
LL13-secure n256p17	2^{13}	256	384	CKKS	6	1	5.03	5.17	$1.10\times$
				GL	4		4.60	4.72	
LL14-secure n512p17	2^{14}	512	576	CKKS	9	1	16.95	8.20	$1.16\times$
				GL	8		14.94	7.09	
HT15-secure n1024p17	2^{15}	1024	1056	CKKS	17	8	18.76	42.13	$1.09\times$
				GL	16		15.88	38.56	

Table 5. Counts and polynomial-payload memory for the StC and hybrid blind-rotation keys. The StC count for GL is shown as the number of small W -rotation keys plus one large conjugate transpose key.

Parameter set	N	Scheme	θ	w	Masked column	HMux	StC keys	StC (GB)	Hybrid blind rotation (GB)	Selected total (GB)
LL13-secure n256p17	2^{13}	CKKS	6	384	744	372	158	0.12	7.90	8.02
		GL	4		848		6 + 1	0.41	8.02	8.43
LL14-secure n512p17	2^{14}	CKKS	9	576	1116	372	190	0.20	10.73	10.93
		GL	8		1504		6 + 1	1.35	11.74	13.09
HT15-secure n1024p17	2^{15}	CKKS	17	1056	2108	372	318	1.00	50.64	51.64
		GL	16		2880		6 + 1	22.57	58.33	80.90

Table 5 measures and counts only the StC keys and the hybrid blind-rotation keys, with the latter comprising the masked-column and HMux keys. The total is the sum of these two memory measurements. We sum the stored 64-bit RNS coefficients and report decimal gigabytes (1 GB = 10^9 bytes). Beyond the two measured categories, the online bootstrapping paths use only relinearization, conjugation, and dense-to-sparse switching keys. We omit these keys because they have the same size in the matched CKKS and GL configurations; no other evaluation-key family is used. The count columns in Table 5 report masked-column ciphertexts, logical HMux branches, and StC automorphism keys, respectively; for GL, the StC count distinguishes the small W -rotation keys from the one large conjugate transpose key.

Our GL bootstrapping reduces the number of logical automorphism keys required by StC: the X -axis uses a native GL plaintext–ciphertext matrix multiplication, and only the W -axis uses a diagonal BSGS transform. However, the large conjugate transpose key used for the X -axis contains n times as many coefficients as a small rotation key. Consequently, GL uses fewer logical StC keys but not necessarily less StC key memory.

The large conjugate transpose key is used by the X -axis inversion. We generate this key at the maximum supported multiplicative depth, so its polynomial payload covers the RNS limbs available at that depth. The n512p17 configuration supports one multiplication level, whereas n1024p17 supports eight. This larger depth coverage explains the sharp increase in GL StC key memory from n512p17 to n1024p17.

5 Related Work

5.1 Homomorphic Matrix Computations

Prior work reduces homomorphic matrix multiplication to a small number of plaintext modular matrix multiplications. Bae *et al.* address plaintext–ciphertext multiplication, while Park extends this approach to ciphertext–ciphertext multiplication for large matrices [2, 26]. These methods benefit from linear algebra

libraries such as OpenBLAS [33] but remain coefficient-oriented CKKS constructions: their matrix layouts are constrained by the underlying ring, and alternating matrix operations with slot-domain SIMD requires repeated CtS and StC conversions. GL instead provides native slot-encoded matrix arithmetic through its multivariate ring, avoiding this representation mismatch [17].

A closely related concurrent CKKS-based matrix scheme uses a *Slots-in-Coefficient* (SinC) encoding to batch many matrix-multiplication instances [10]. This construction supports small and medium matrix dimensions without tying the matrix dimension or batch size directly to the ring dimension, and it substantially improves batched matrix-multiplication performance over earlier reduction-based methods. SinC therefore addresses an important dimensional constraint of those methods, while retaining the coefficient-oriented representation limitation described above. Our work provides bootstrapping for GL’s native matrix encoding.

5.2 Bootstrapping

CKKS bootstrapping. Classical CKKS bootstrapping research primarily targets improvements in precision, latency, and multiplicative depth within the polynomial-EvalMod paradigm [4–6, 18]. SHIP instead targets a shallower and more parallelizable regime, and its low-depth design objective is the starting point for our work [7]. Whereas SHIP operates on standard CKKS ciphertexts, our construction adapts this objective to the multivariate GL slice algebra while preserving GL’s matrix-native representation.

Cheon *et al.* [12] provide a multidimensional bootstrapping construction for tensor-style CKKS packing, which the GL paper identifies as a possible starting point for GL bootstrapping [17]. That construction extends the classical polynomial-approximation regime to multidimensional CKKS and requires rotation material for its linear transforms. Our work instead targets native (X, W) GL slices and the low-depth SHIP regime. The two approaches therefore differ in both their underlying representation and their depth objective.

Batched CKKS bootstrapping. A recent line of work focuses on improving CKKS bootstrapping throughput by batching ciphertexts. In this direction, Cheon *et al.* introduce MRFHE, a CKKS-type scheme over a mixed-radix cyclotomic ring with degree $N = 2^{w_2}3^{w_3}$ [8]. Its CRT representation exposes the slots as an $n_1 \times n_2$ matrix, with $n_1 n_2 = N/2$, and factors the global DFT into radix-2 and radix-3 transforms of the form

$$\mathbf{D}_1 \mathbf{M} \mathbf{D}_2^T.$$

For a batch of ciphertexts, MRFHE packs the inputs into two auxiliary extension-ring representations and evaluates the left and right DFT factors through reduction-based plaintext matrix multiplication. This replaces the rotation-heavy transform for each ciphertext with an $O(1)$ amortized number of key-switching operations per ciphertext. Their proof-of-concept experiments report $1.42\times$ – $1.73\times$ throughput improvements for the complete bootstrapping pipeline with batches of 64–256 ciphertexts.

MRFHE provides a two-dimensional slot layout with independent row and column rotations and supports reduction-based batched matrix multiplication. Unlike GL, it does not provide a native matrix product on matrix-encoded ciphertexts.

Concurrent GL bootstrapping. Concurrent work by Crockett *et al.* [13] develops an alternative, classical EvalMod-based bootstrapping construction for GL. Their X -axis linear transform, like ours, uses native GL plaintext–ciphertext matrix multiplication and a conjugate transpose. For the W axis, however, they introduce an auxiliary variable V and work over

$$R_{\text{aux}} = \mathbb{Z}[I, X, W, V] / \langle I^2 + 1, X^n - I, \Phi_p(W), \Phi_p(V) \rangle.$$

Given slice ciphertexts $\text{ct}_j = (b_j, a_j)$ modulo Q_L , they form one packed ciphertext $\text{ct}^{\text{ext}, V} \in (R_{Q_L}^{\text{aux}})^2$, where $R_{Q_L}^{\text{aux}} = R_{\text{aux}} / Q_L R_{\text{aux}}$:

$$\text{ct}^{\text{ext}, V} = \left(\sum_j b_j V^j, \sum_j a_j V^j \right).$$

The coefficient of V^j therefore stores the j -th slice ciphertext; because the secret $s(X, W)$ is independent of V , this packing preserves decryption. They next apply a conjugate transpose that conjugates the Gaussian coefficients, inverts the X -axis, and exchanges the inverted W - and V -axes. Because this transformation changes the secret-key representation, they key-switch the result back to the original secret $s(X, W)$. To evaluate the W -axis DFT, they encode its matrix in a polynomial with a second temporary variable U , multiply the transposed ciphertext by this polynomial, and apply Tr_U . At the slot level, this trace computes the inner-product sum over the U -index and removes U from the result; it does not remove the packing variable V . After reversing the conjugate transpose, they extract each coefficient of V^j to recover the transformed slice ciphertexts. Their optimized ModRaise/CtS procedure uses three distinct switching keys: one small dense-to-sparse key over the (X, W) ring, one big conjugate transpose key over the full GL ring, and one big conjugate transpose key over R_{aux} . The full-ring key is applied once and the auxiliary-ring key twice, giving three big key-switching operations. This count excludes the relinearization material used by EvalMod.

For the W -axis key material, their construction requires one big switching key over R_{aux} . A polynomial over the (X, W) RLWE ring has $n\varphi(p)$ Gaussian coefficients, whereas a polynomial over R_{aux} has $n\varphi(p)^2$. Thus, for the same RNS basis and gadget decomposition, this auxiliary key has the coefficient payload of $\varphi(p)$ small RLWE-ring switching keys. Our W -axis StC instead uses a BSGS decomposition with $O(\sqrt{\varphi(p)})$ small W -rotation keys and requires no auxiliary-ring key. This is a comparison of the W -axis key payloads; both constructions use separate key material for their X -axis transforms. In their reported instance $(n, p) = (64, 257)$, R_{aux} contains $64 \cdot 256 \cdot 256$ coefficients; the paper reports that operations in this ring are approximately eight times slower than operations in its $64 \cdot 64 \cdot 256$ -coefficient three-variable ring, partly because the prime-length dimensions use Rader transforms. In this work, we algorithmically optimize the Rader NTT to substantially reduce its arithmetic count and close the performance gap with the standard one-dimensional radix-2 Cooley–Tukey NTT used by CKKS, as reflected by our experiments.

The two constructions also target different bootstrapping regimes. Crockett *et al.* optimize the linear transforms in the classical StC–ModRaise–CtS–EvalMod pipeline, in which polynomial EvalMod remains the dominant source of multiplicative depth and runtime. Evaluating this polynomial requires a long modulus chain; under a fixed security target, accommodating that chain requires a larger RLWE ring dimension. Consequently, their EvalMod-based construction is feasible only at the large dimensions $\log_2 N \in \{15, 16\}$ considered in their work. Our construction instead evaluates decryption in the exponent, bypassing the depth imposed by polynomial EvalMod and enabling the smaller dimensions $\log_2 N \in \{13, 14\}$. To the best of our knowledge, this is the first GL bootstrapping construction to enter this low-depth regime.

6 Conclusion

We presented a low-depth bootstrapping framework that operates directly on GL ciphertexts. The construction combines a grouped GL StC transform with a SHIP-style half-bootstrap derived for the (X, W) slice algebra, while preserving GL’s native matrix representation. The construction uses X -only hidden HMux selection, algebraically reduces the number of online plaintext encodings, and accelerates the W -axis transform using a fused Rader NTT. Under matched parameter sets targeting 128 bits of security, we demonstrate six-level complex bootstrapping for $\log_2 N \in \{13, 14, 15\}$. Across the evaluated configurations, GL achieves a $2.55\times$ – $3.12\times$ speedup in amortized per-slice StC latency and a $1.09\times$ – $1.16\times$ speedup in amortized per-slice complex-bootstrap latency over the matched CKKS baselines.

Directions for future research include evaluating alternative prime-length NTT implementations, such as Bluestein’s algorithm, and investigating whether Winograd NTT algorithms can reduce the number of modular multiplications and improve latency. Further work includes applying GL and the proposed bootstrapping framework to private neural-network inference, including transformer models, and exploring GPU acceleration for the computational primitives that dominate GL bootstrapping and matrix multiplication.

Acknowledgments

This work has been supported by NSF Award #2239334.

References

1. Aikata Aikata and Sujoy Sinha Roy. Secure and efficient outsourced matrix multiplication with homomorphic encryption. In *Progress in Cryptology – INDOCRYPT 2024*, pages 51–74. Springer, 2024.
2. Youngjin Bae, Jung Hee Cheon, Guillaume Hanrot, Jai Hyun Park, and Damien Stehlé. Plaintext-ciphertext matrix multiplication and FHE bootstrapping: Fast and fused. In *Advances in Cryptology – CRYPTO 2024*, pages 387–421. Springer, 2024.
3. Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeon Kim, Bryan Kumara, Changmin Lee, et al. Security guidelines for implementing homomorphic encryption. *IACR Communications in Cryptology*, 1(4), 2025.
4. Jean-Philippe Bossuat, Christian Mouchet, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *Advances in Cryptology – EUROCRYPT 2021*, pages 587–617. Springer, 2021.
5. Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2019*, pages 34–54. Springer, 2019.
6. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology – EUROCRYPT 2018*, pages 360–384. Springer, 2018.
7. Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. SHIP: A shallow and highly parallelizable CKKS bootstrapping algorithm. In *Advances in Cryptology – EUROCRYPT 2025*, pages 398–428. Springer, 2025.
8. Jung Hee Cheon, Seungwan Hong, Minsik Kang, Jonghyun Kim, Taeseong Kim, Changmin Lee, and Junho Lee. MRFHE: Mixed-radix fully homomorphic encryption with better batch bootstrapping. *Cryptology ePrint Archive*, Paper 2026/853, 2026.
9. Jung Hee Cheon, Minsik Kang, Taeseong Kim, Junyoung Jung, and Yongdong Yeo. Batch inference on deep convolutional neural networks with fully homomorphic encryption using channel-by-channel convolutions. *IEEE Transactions on Dependable and Secure Computing*, 22(2):1674–1685, 2025.
10. Jung Hee Cheon, Minsik Kang, and Junho Lee. Fast batch matrix multiplication in ciphertexts. *Cryptology ePrint Archive*, (2025/1957), 2025.
11. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437. Springer, 2017.
12. Jung Hee Cheon, Andrey Kim, and Donggeon Yhee. Multi-dimensional packing for HEAAN for approximate matrix arithmetics. *Cryptology ePrint Archive*, (2018/1245), 2018.
13. Eric Crockett, Craig Gentry, Hyojun Kim, Yeongmin Lee, and Yongwoo Lee. Efficient bootstrapping in fully homomorphic encryption for matrix arithmetic. *Cryptology ePrint Archive*, Paper 2026/956, 2026.
14. Desilo Inc. The DESILO FHE library. <https://fhe.desilo.dev/latest/>, 2026. Accessed: July 13, 2026.
15. Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, pages 169–178, 2009.
16. Craig Gentry, Shai Halevi, and Nigel P. Smart. Better bootstrapping in fully homomorphic encryption. In *Public Key Cryptography – PKC 2012*, pages 1–16. Springer, 2012.
17. Craig Gentry and Yongwoo Lee. Fully homomorphic encryption for matrix arithmetic. *Cryptology ePrint Archive*, (2025/1935), 2025.
18. Kyoohyung Han and Dohyeon Ki. Better bootstrapping for approximate homomorphic encryption. In *Topics in Cryptology – CT-RSA 2020*, pages 364–390. Springer, 2020.
19. Lucjan Hanzlik, Yang Zhang, Kathrin Grosse, Ahmed Salem, Maximilian Augustin, Michael Backes, and Mario Fritz. MLCapsule: Guarded offline deployment of machine learning as a service. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 3300–3309, 2021.
20. Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. EvalRound algorithm in CKKS bootstrapping. In *Advances in Cryptology – ASIACRYPT 2022*, pages 161–187. Springer, 2022.
21. Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, et al. Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. *IEEE Access*, 10:30039–30054, 2022.
22. Long Li, Jianxin Lai, Peng Yuan, Tianxiang Sui, Yan Liu, Qing Zhu, Xiaojing Zhang, Linjie Xiao, Wenguang Chen, and Jingling Xue. ANT-ACE: An FHE compiler framework for automating neural network inference. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, pages 193–208, 2025.
23. Lawrence Lim, Vikas Kalagi, Divyakant Agrawal, and Amr El Abbadi. Tricycle: Private transformer inference with tricyclic encodings. *Cryptology ePrint Archive*, (2025/1200), 2025.

24. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 1–23. Springer, 2010.
25. Alexander May. How to meet ternary LWE keys. In *Advances in Cryptology – CRYPTO 2021*, volume 12826 of *Lecture Notes in Computer Science*, pages 701–731. Springer, 2021.
26. Jai Hyun Park. Ciphertext-ciphertext matrix multiplication: Fast for large matrices. In *Advances in Cryptology – EUROCRYPT 2025*, pages 153–180. Springer, 2025.
27. Charles M. Rader. Discrete fourier transforms when the number of data samples is prime. *Proceedings of the IEEE*, 56(6):1107–1108, 1968.
28. Reza Shokri and Vitaly Shmatikov. Privacy-preserving deep learning. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 1310–1321, 2015.
29. Zejiu Tan, Junping Wan, Zoe L. Jiang, Jingjing Fan, Man Ho Au, and Siu Ming Yiu. High-precision homomorphic modular reduction for CKKS bootstrapping. In *Information Security and Privacy – ACISP 2025*, pages 232–251. Springer, 2025.
30. Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction APIs. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 601–618. USENIX Association, 2016.
31. Runhua Xu, Nathalie Baracaldo, and James Joshi. Privacy-preserving machine learning: Methods, challenges and directions. *arXiv preprint*, (arXiv:2108.04417), 2021.
32. Jiawen Zhang, Jian Liu, Lipeng He, Xinpeng Yang, Wen-jie Lu, Yinghao Wang, Kejia Chen, Xiaoyang Hou, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. *Cryptology ePrint Archive*, (2024/136), 2024.
33. Xianyi Zhang, Qian Wang, and Yunquan Zhang. Model-driven level 3 BLAS performance optimization on Loongson 3a processor. In *2012 IEEE 18th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 684–691. IEEE, 2012.